

SYSTEM ANALYST

presented by Ayub

UML: Sequence Diagram

Где мы находимся

Use Case диаграмма показала **ЧТО** умеет система — список функций глазами пользователя.

Sequence Diagram покажет **КАК** именно это работает — порядок сообщений между участниками.

Инструмент	Отвечает на
Use Case	ЧТО умеет система
Sequence Diagram	КАК выполняется каждый сценарий

Это главный язык общения SA с разработчиками.

Ты отправил другу сообщение в WhatsApp.

Нажал "Отправить" — и через 0.3 секунды друг получил.

Как думаешь: что происходит за эти 0.3 секунды? Кто с кем общается?

Подсказка: твой телефон, интернет, серверы WhatsApp, телефон друга.

Вот как это выглядит на диаграмме

Sequence Diagram — это временная шкала.

Сверху вниз = время идёт вниз.

Каждый участник разговора — вертикальная линия. Сообщения — горизонтальные стрелки.

- Участники сверху
- Время идёт вниз
- Стрелки = сообщения
- Пунктир = ответ

Айб

! "

эт! Как дела?"

ю, ты свободен сегодня?"

о сколько?"

Айб

Айб

Зачем SA рисует Sequence Diagram

- **Проектирование API:** какие запросы, в каком порядке, что возвращается
- **Описание сложных сценариев:** авторизация, оплата, регистрация
- **Передача разработчикам:** "вот как это должно работать — вопросы?"
- **Поиск узких мест:** где система ждёт? кто блокирует поток?
- **Согласование с бизнесом:** заказчик смотрит и говорит "да, именно так"

*SA рисует диаграмму **до** начала разработки — чтобы найти проблемы в логике, пока их ещё дёшево исправить.*

Этап 2: Элементы диаграммы

Когда ты заказываешь еду через приложение — кто или что участвует в этом процессе?

Назови всех: от твоего телефона до курьера у двери.

Participant — участник

Participant — любой, кто участвует в сценарии.

Рисуется как прямоугольник **сверху диаграммы**. Бывает:

- **Actor** — человек или внешняя система (иконка человечка)
- **Participant** — компонент системы (прямоугольник)
- **Database** — база данных (цилиндр)



Правило: не более 5-7 участников на одной диаграмме. Больше — читать невозможно.

Lifeline — линия жизни

Lifeline — пунктирная вертикальная линия вниз от каждого участника.

Показывает: участник **существует** в этот момент времени.

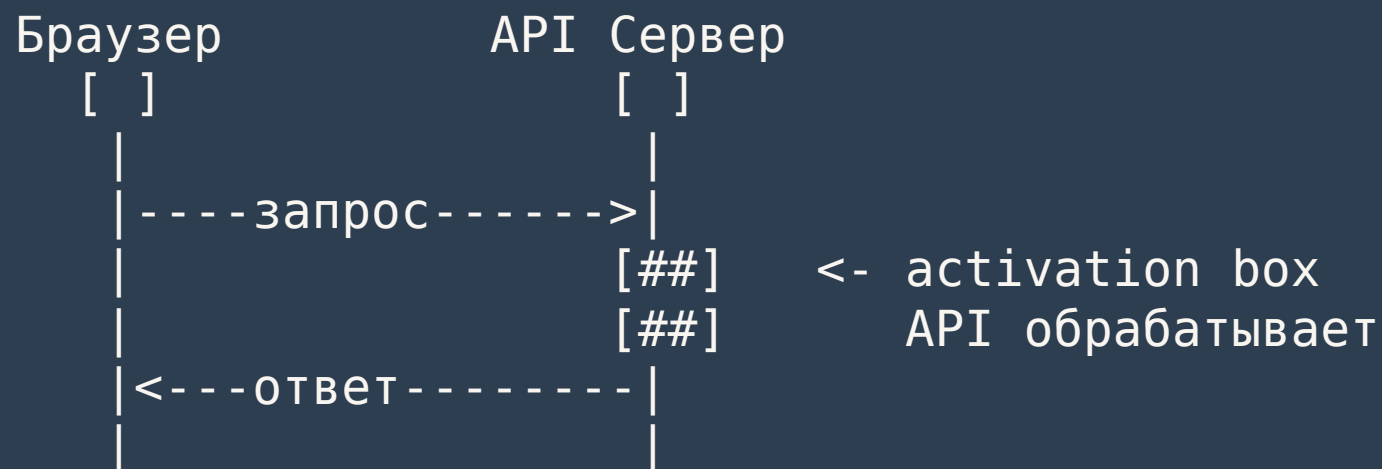


- Пока линия есть — участник существует
- Линия прерывается — участник уничтожен (редко используется)

Activation Box — блок активации

Activation Box — прямоугольник на линии жизни.

Показывает: участник сейчас **занят выполнением операции**.



- Начало box = получил вызов
- Конец box = вернул ответ
- Помогает видеть: кто занят и как долго

Messages — сообщения

Стрелки между участниками. Тип стрелки = тип взаимодействия.

Тип	Стрелка	Когда
Синхронный	-> сплошная	Отправил и ждёт ответа
Асинхронный	->> открытая	Отправил и не ждёт
Ответ	--> пунктир	Возврат результата
Self-call	петля	Метод вызывает сам себя

UML: Sequence Diagram



Module Check 1

1. **Что показывает** ось Y (вертикаль) на Sequence Diagram?
2. **Что такое** Activation Box и что он означает?
3. **В чём разница** между синхронным и асинхронным сообщением?
4. **Где рисуется** Participant?
5. **Сколько максимум** участников рекомендуется на одной диаграмме?

Этап 3: Combined Fragments

Как показать на диаграмме:

"Если пароль верный — пустить в кабинет, иначе — показать ошибку"

Или: "Проверять новые сообщения каждые 30 секунд"

Как записать условие и повтор в диаграмме?

Combined Fragments — рамки управления потоком

Специальные прямоугольники-рамки, которые обозначают логику.

Фрагмент	Смысл	Аналог в коде
alt	если / иначе	if / else
loop	повторение	for / while
opt	опционально	if без else
ref	ссылка на другую диаграмму	вызов функции

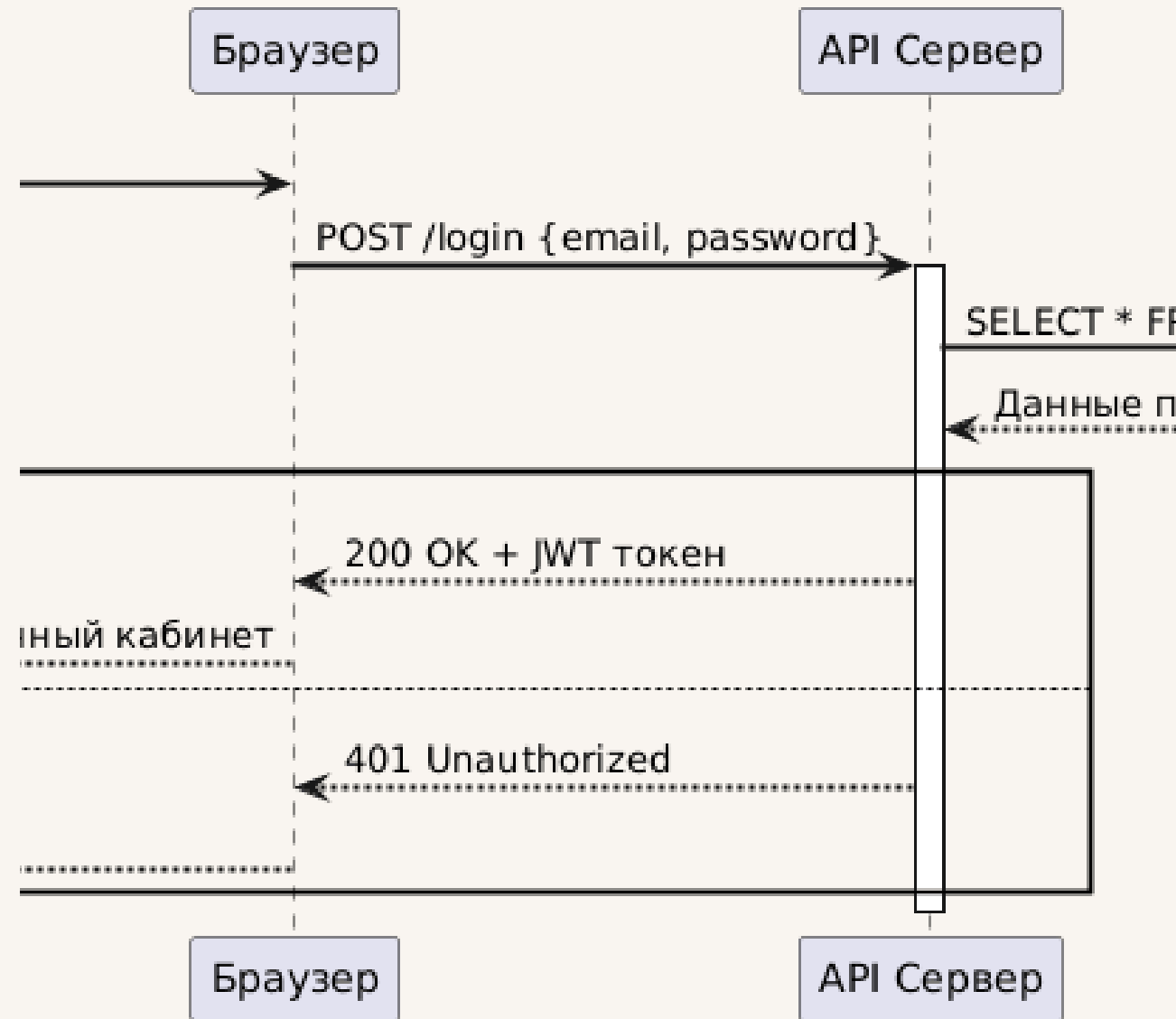
Рамка рисуется вокруг сообщений. В верхнем левом углу — название фрагмента.

alt — авторизация

Главный пример: **вход в систему**. Два пути — верный пароль и нет.

Рамка `alt` разделена пунктирной линией — верхняя и нижняя секция.

- `alt` — рамка с двумя секциями
- Верхняя: условие выполнено
- Нижняя: `else` — не выполнено
- Условие пишется в `[скобках]`



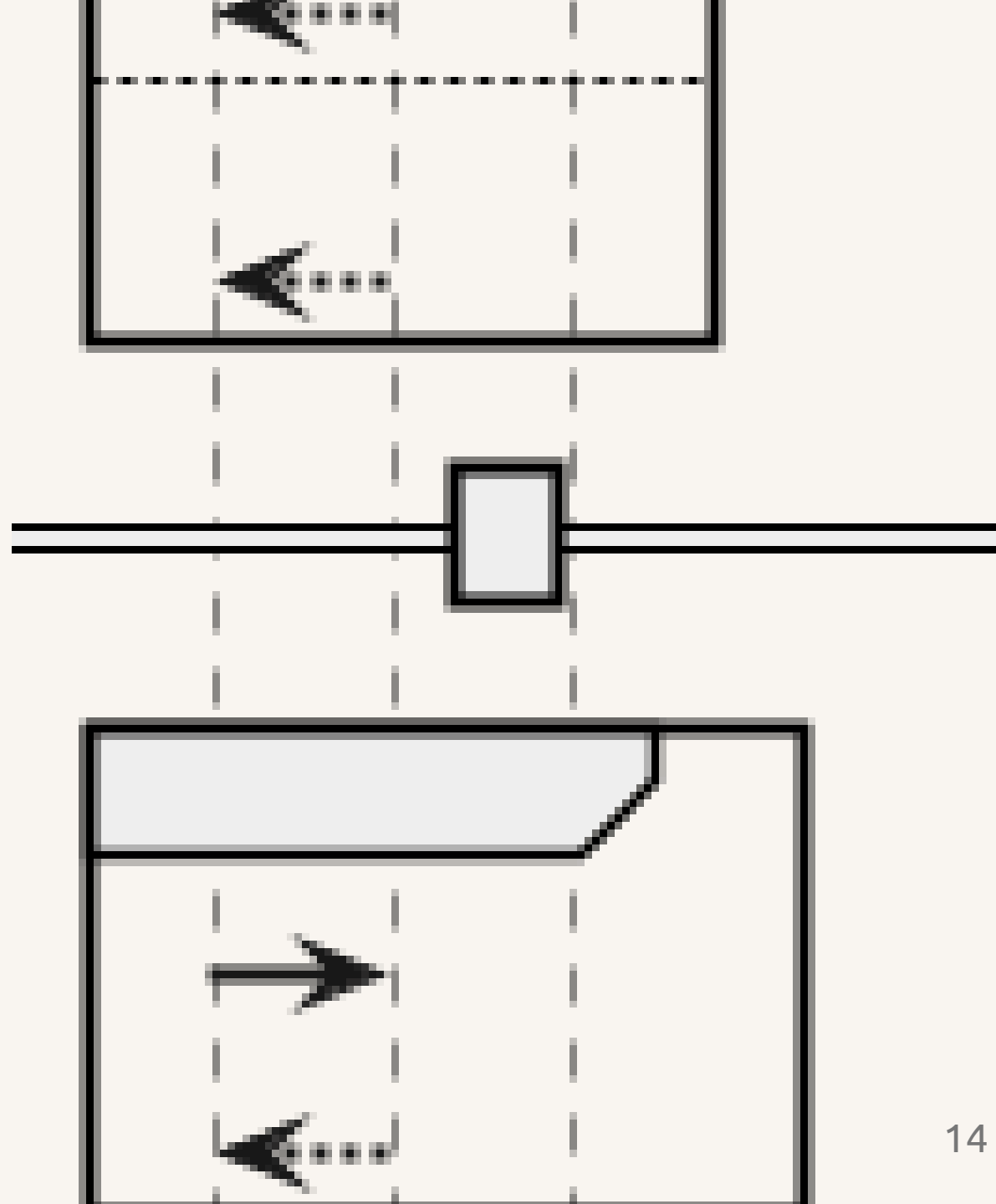
loop и opt

loop — повторять блок сообщений пока условие истинно.

opt — выполнить блок только если условие выполнено (нет "иначе").

```
loop [каждые 30 сек]
  App -> Server : проверить сообщения
  Server --> App : список
end

opt [уведомления включены]
  Server ->> Push : отправить уведомление
end
```



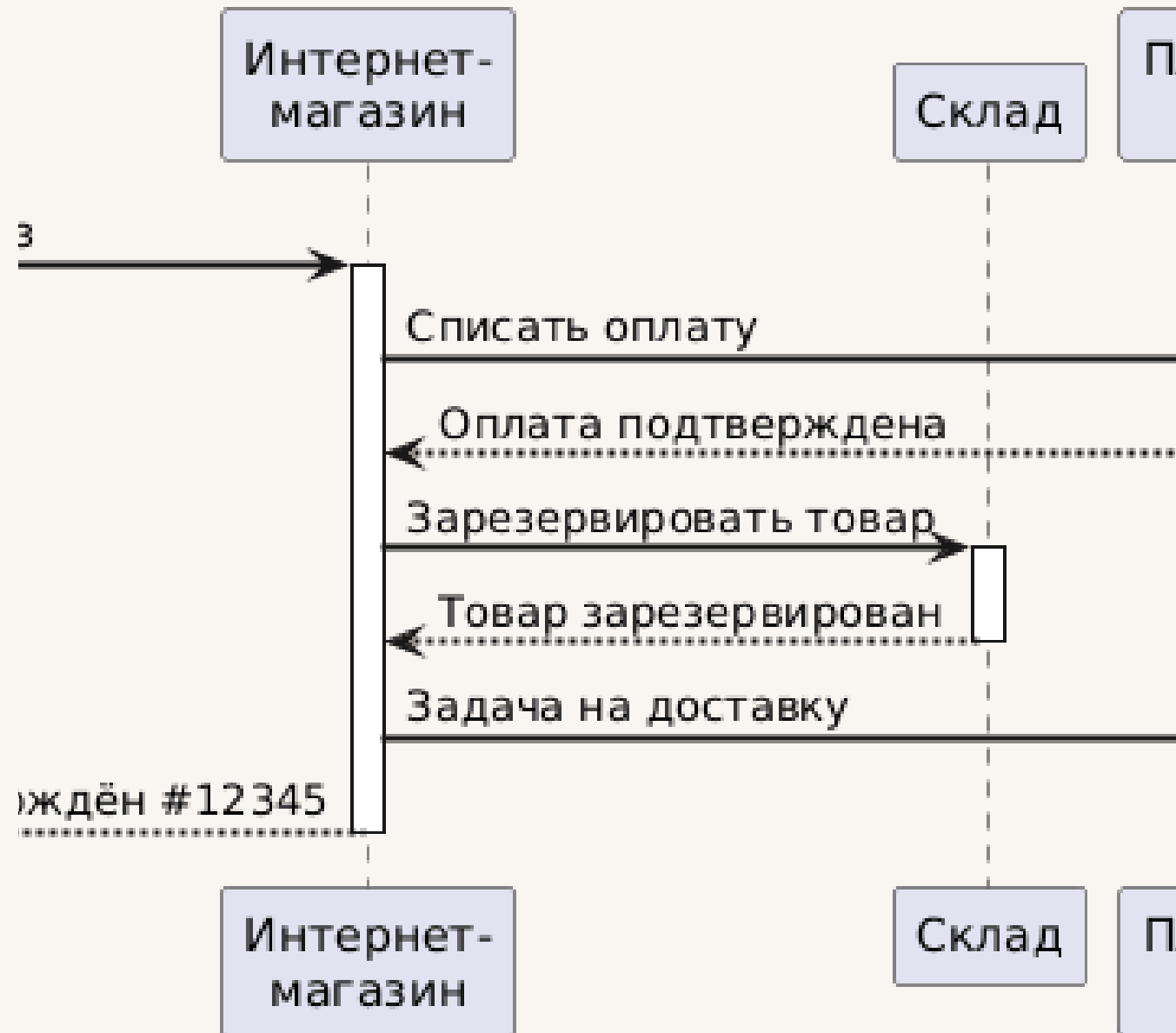
Практический пример: заказ в интернет-магазине

Посмотрим на полный сценарий с несколькими участниками.

Разбор:

- Синхронные вызовы к Payment — **ждём** подтверждения
- Синхронные вызовы к Warehouse — **ждём** резервирования
- Асинхронный к Courier — **не ждём**, задача в фоне

UML: Sequence Diagram



Микропрактика: 7 минут

Нарисуй Sequence Diagram для сценария "Зарегистрироваться на сайте".

Участники: Пользователь, Браузер, API, База данных, Email-сервис

Покажи:

- Ввод данных → проверка → сохранение → письмо подтверждения
 - Используй `alt` для случая "email уже занят"
 - Отправка письма — синхронная или асинхронная?
7. Browser --> User: "Регистрация прошла"
- Акцент: email отправляется асинхронно — пользователю не нужно ждать пока письмо дойдёт
 - ~7 мин работа, 3 мин разбор

Peer Review: проверь у соседа

Поменяйтесь листами. 3 минуты на разбор:

- Время идёт **вниз**?
- Ответные сообщения — пунктирная стрелка `-->` ?
- Activation boxes там где участник занят?
- Email-сервис — асинхронный вызов `->>` ?
- `alt` рамка разделена пунктиром на два пути?

Верни лист с одной правкой и одной похвалой.

Этап 4: Типичные ошибки

**Смотришь на чужую диаграмму — стрелки в обе стороны
сплошные, нет рамок, нет пунктира.**

Что именно не так и почему это мешает читать?

Ошибка 1 и 2

Ошибка 1: ответные сообщения — сплошная стрелка

- Нельзя: `Browser <- API` сплошная стрелка как запрос
- Правильно: ответ всегда пунктирный `Browser <- - API`
- Почему: читатель не понимает, что это ответ, а не новый запрос

Ошибка 2: нет activation boxes

- Нельзя: нарисовал запросы без activation
- Правильно: показывай кто занят и как долго
- Почему: непонятно где узкое место, сколько ждёт клиент

Ошибка 3 и 4

Ошибка 3: слишком много деталей

- Нельзя: каждый SQL-запрос, каждый HTTP-заголовок
- Правильно: Сохранить пользователя, а не `INSERT INTO users VALUES (...)`
- Уровень абстракции должен быть одинаковым на всей диаграмме

Ошибка 4: только happy path

- Нельзя: показать только успешный сценарий
- Правильно: всегда добавляй `alt` с ошибочным случаем
- SA ценен тогда, когда видит edge cases раньше разработчика

Диаграмма без обработки ошибок — это полуправда.

Module Check 2

1. **Зачем** нужен `alt` фрагмент?
2. **Чем** `opt` **отличается** от `alt` ?
3. **Когда использовать** асинхронное сообщение `->>` ?
4. **Что такое** Lifeline?
5. **Нарисуй мысленно:** вход через Google (OAuth). Какие участники?
Порядок?

Этап 5: Sequence Diagram в работе SA

Где именно ты будешь рисовать эти диаграммы на проекте?

- **API specification**: какой endpoint, какой запрос, какой ответ, в каком порядке
- **SRS**: раздел "Описание сценариев" — каждый Use Case детализируется через Sequence
- **Tech brief**: документ для разработчика — "вот как это должно работать"
- **Ревью архитектуры**: показать команде, обсудить до написания кода

| Диаграмма передаётся разработчику **вместо** словесного описания.

Связь с другими артефактами SA

Каждый инструмент отвечает на свой вопрос.

Инструмент	Отвечает на
Use Case Diagram	ЧТО умеет система
Sequence Diagram	КАК выполняется Use Case
Class Diagram	ИЗ ЧЕГО состоит система
BPMN	КАК идёт бизнес-процесс (с ролями)

Sequence + Use Case = полная картина поведения системы.

Инструменты для рисования

Инструмент	Плюсы	Минусы
draw.io	Бесплатно, визуально	Медленно руками
PlantUML	Текстом, быстро, в git	Нужно знать синтаксис
Mermaid	Встраивается в Confluence/GitHub	Ограниченный синтаксис
Lucidchart	Красиво, командная работа	Платно

Совет: начинай с draw.io для понимания, потом PlantUML или Mermaid для скорости.

Чек-лист хорошей Sequence Diagram

- [] Время идёт **сверху вниз**
- [] У каждого участника есть Lifeline
- [] Activation boxes показывают кто занят
- [] Ответные сообщения — **пунктирные** `-->`
- [] Есть обработка ошибок (`alt` с error case)
- [] Уровень абстракции одинаковый по всей диаграмме
- [] Не более **5-7 участников**
- [] Асинхронные вызовы обозначены `->>` где нужно

Если все галочки стоят — разработчик прочитает без вопросов.

Module Check финальный

1. **В чём главное отличие** Sequence Diagram от Use Case Diagram?
2. **Нарисуй мысленно:** отправка сообщения в Telegram. Участники?
Порядок?
3. **Зачем SA рисует** Sequence Diagram **до** начала разработки?
4. **Что показывает** Activation Box?
5. **Почему ответы** рисуются пунктирной стрелкой?

Итого

Sequence Diagram показывает порядок сообщений между участниками во времени — это язык общения SA с разработчиками.

Что ты теперь умеешь:

- Читать и рисовать Sequence Diagram
- Обозначать участников, lifeline, activation boxes
- Показывать sync/async сообщения
- Использовать `alt`, `loop`, `opt` фрагменты
- Передавать диаграмму разработчику как спецификацию

Что дальше

OOP Basics — как устроены объекты, которые мы видели в Sequence Diagram

Class Diagram — структура системы: что за объекты, как они связаны между собой

Sequence + Class = полное понимание системы:

- Sequence показывает как объекты общаются во времени
- Class показывает из чего они состоят

Ты уже умеешь описывать поведение системы. Следующий шаг — описать её структуру.